

Final Project Report: Mocha: Momentum for Overcoming Challenges in Heterogeneous Aggregation

Ansel Erol
Georgia Institute of Technology
Atlanta, GA, USA
aerol3@gatech.edu

Mohamed Aamir
Georgia Institute of Technology
Atlanta, GA, USA
maamir7@gatech.edu

Alireza Moradi
Georgia Institute of Technology
Atlanta, GA, USA
amoradi30@gatech.edu

Michael Alekseychyk
Georgia Institute of Technology
Atlanta, GA, USA
malekseychyk3@gatech.edu

Abstract

Federated Learning (FL) enables collaborative model training on decentralized data while preserving privacy, but faces challenges in convergence speed and efficiency, especially with heterogeneous (non-IID) data distributions across clients. This research addresses these challenges through a systematic comparative analysis of five advanced momentum-based optimization methods integrated into the Federated Averaging (FedAvg) framework: traditional Polyak momentum, Nesterov Accelerated Gradient (NAG), Quasi-Hyperbolic Momentum (QHM), Rectified Adam (RAdam), and Accelerated SGD (AccSGD). While Polyak and Nesterov have seen prior limited study in FL, this work provides the first known systematic evaluation of QHM, RAdam, and AccSGD in this setting. Using PyTorch and the Flower simulation platform, we benchmarked these optimizers on CIFAR-10 and CIFAR-100 datasets, partitioned across 10 clients under both IID and non-IID (Dirichlet $\alpha = 0.5$) conditions, training a 2-layer CNN over 50 communication rounds. Our experiments consistently revealed a performance hierarchy where Polyak momentum (with $m = 0.1$) achieved the fastest convergence and highest final accuracy, followed closely by RAdam. Nesterov, QHM, and AccSGD demonstrated comparatively slower convergence across all tested scenarios. The results also underscore the significant negative impact of non-IID data on FL performance for all optimizers investigated. Github repo can be found here: <https://github.com/A-K-Erol/7643-fl-momentum>.

1. Introduction

Federated Learning (FL) has emerged as a promising paradigm for training machine learning models on decentralized data while preserving privacy [7]. In this approach, models are trained locally on edge devices, with parameter updates aggregated and refined on a centralized server before being redistributed. While this methodology has shown considerable promise, the convergence rate and training efficiency of federated learning systems remain significant challenges due to reasons such as:

1. Insufficient amount of data in individual devices: Even though we may have enough data overall, its distributed nature makes it challenging to learn insights and patterns in the entire dataset.
2. Heterogeneous data: Data in local edge devices may be non-IID.

This research proposes a comprehensive comparative analysis of advanced momentum-based optimization methods in the context of federated learning, specifically examining:

1. Traditional Polyak momentum
2. Nesterov momentum
3. Quasi-Hyperbolic Momentum (QHM)
4. Rectified Adam (RAdam)
5. ACC SGD (Accelerated SGD)

Notably, while the first two methods have been explored in federated contexts [5, 6], this work represents the first systematic investigation of QHM, RAdam, and ACC SGD in

federated learning environments. These advanced optimizers have shown promising results in centralized deep learning but remain unexplored in federated scenarios, presenting a significant research opportunity.

This problem is important because federated learning can be very useful in applications where data privacy and data decentralization are crucial. Some of these areas include mobile devices, healthcare, and industrial engineering [1]. The acceleration of federated learning convergence is crucial for several reasons.

First, momentum-based methods have demonstrated substantial success in accelerating convergence in both traditional deep learning training and initial federated learning implementations. However, the federated learning community has primarily focused on basic momentum and Nesterov approaches, leaving significant potential unexplored.

Second, the challenge of heterogeneous data distribution across nodes makes optimization particularly crucial in federated learning. When client devices contain non-IID (Independent and Identically Distributed) data, traditional optimization methods often struggle to find optimal solutions efficiently.

Lastly, advanced optimizers like QHM offer adaptive momentum adjustment capabilities that could better handle local-global optimization conflicts. RAdam’s adaptive learning rate rectification could provide more stable training across heterogeneous clients, while ACC SGD’s momentum-based variance reduction technique might improve convergence in non-convex federated optimization landscapes.

2. Related Works

[3] presents a federated learning approach developed to be robust to non-IID data distribution across devices. Accounting for non-IID distributions and unbalanced datasets in federated learning clients is necessary, as the amount and distribution of data on a particular client device depends on its usage. The authors show how techniques such as FederatedAveraging (FedAvg) and FederatedSGD can be used to train high-quality models with minimal communication.

[4] introduces an improvement on the FederatedAveraging algorithm by adding periodic server-side averaging. The experiments performed by the authors show that server averaging can achieve faster convergence to a target accuracy and reduce client computational costs. This adaptation has also been shown to find better generalizing optima and increase the effectiveness of FL through the recycling of old clients.

[5] proposes a decentralized federated averaging algorithm with momentum (DFedAvgM). In this method, clients perform SGD with momentum and communicate only with

their neighbors. This reduces reliance on a central server, improving privacy and making the system less susceptible to attacks. The paper also explores the theoretical convergence of DFedAvgM and its quantized version under general non-convex assumptions.

[6] studies federated learning with Nesterov Accelerated Gradient (NAG). The authors propose FedNAG, which incorporates NAG momentum and model aggregation on the server, as well as NAG in each client. Their analysis shows that, as long as the learning step size is small enough, FedNAG outperforms FedAvg. Experimental results show that FedNAG increases learning accuracy by 3–24% and reduces total training time by 11–70%.

3. Methodology

This section describes our approach to enhancing federated learning (FL) optimization by integrating advanced momentum-based optimizers into the standard Federated Averaging (FedAvg) framework. We implement and benchmark these methods using PyTorch and the Flower FL simulation platform, leveraging Ray for distributed orchestration.

Data Processing and Partitioning

We conduct experiments on the CIFAR-10 and CIFAR-100 datasets. Datasets are normalized using standard channel-wise statistics and split among clients using two strategies:

- **IID:** Data is randomly shuffled and evenly partitioned across clients.
- **Non-IID:** Clients receive samples biased toward specific classes to emulate real-world heterogeneity.

Data augmentation techniques such as random cropping and horizontal flipping are optionally employed to enhance model generalization.

Model Architecture

Each client trains a compact convolutional neural network suitable for edge devices, consisting of two or three convolutional layers followed by fully connected layers. All clients share the same model architecture to ensure parameter aggregation is feasible across devices. The models are implemented in PyTorch and trained locally on each client.

Optimization Techniques

We implement several optimizers within the federated learning loop:

- SGD with Polyak Momentum
- Nesterov Accelerated Gradient (NAG)

- Quasi-Hyperbolic Momentum (QHM)
- Rectified Adam (RAdam)
- Accelerated SGD (AccSGD)

The QHM optimizer is implemented from scratch based on the definition in [2]. It interpolates between momentum and plain SGD updates through a blending parameter ν . RAdam and AccSGD are incorporated using external libraries and adapted to the federated learning workflow.

Each optimizer modifies the local training step on clients before model aggregation. For QHM, the local update rule follows:

$$\begin{aligned} m_t &= \beta m_{t-1} + (1 - \beta) \nabla f_i(x_t), \\ g_t &= (1 - \nu) \nabla f_i(x_t) + \nu m_t, \\ x_{t+1} &= x_t - \eta g_t, \end{aligned}$$

where ν is the blending parameter between the gradient $\nabla f_i(x_t)$ and the momentum buffer m_t .

Federated Learning Framework

We use the Flower framework to simulate a federated system comprising multiple clients and a central server. Each communication round proceeds as follows:

1. The server selects a subset of clients.
2. Selected clients download the global model.
3. Each client trains locally using the chosen optimizer for a fixed number of local epochs.
4. Clients upload updated model parameters.
5. The server aggregates updates using weighted averaging (FedAvg).

The Flower simulation backend is configured with Ray, enabling parallel client training across virtual actors.

Training Procedure

For each experiment, we simulate federated training for a fixed number of communication rounds (e.g., 50 rounds). In each round:

- A fixed fraction of clients (e.g., 10 out of 100) participate.
- Each client trains locally for a small number of epochs (e.g., 5) with a fixed mini-batch size.
- After local training, models are sent back to the server and aggregated.

Pseudocode outlining the federated optimization workflow:

Algorithm 1 Federated Learning with Advanced Momentum

```

1: Initialize global model parameters  $x^0$ 
2: for each round  $t = 1$  to  $T$  do
3:   Server selects client subset  $S_t$ 
4:   for each client  $i \in S_t$  in parallel do
5:     Client downloads  $x^t$ 
6:     Client trains locally using optimizer (SGD,
       NAG, QHM, etc.)
7:     Client uploads updated parameters  $x_i^t$ 
8:   end for
9:   Server aggregates updates:  $x^{t+1} = \sum_{i \in S_t} \frac{n_i}{n} x_i^t$ 
10: end for

```

Evaluation Metrics

We evaluate the performance of each optimizer across several criteria:

- **Test Accuracy:** Final model accuracy on a centralized holdout test set.
- **Convergence Speed:** Number of communication rounds needed to reach target accuracy thresholds.
- **Robustness to Non-IID Data:** Performance degradation when training on heterogeneous client data.
- **Communication Cost:** Total number of model update transmissions required to reach convergence.

Results are recorded automatically at each communication round and stored for analysis.

4. Data

Dataset Source and Composition. This study utilized either the CIFAR-10 or CIFAR-100 benchmark dataset. Both consist of 32×32 pixel color images. CIFAR-10 comprises 10 distinct object classes, while CIFAR-100 contains 100 classes hierarchically grouped into 20 superclasses. These standard datasets were employed to evaluate the proposed methods in controlled image classification tasks.

Federated Data Partitioning. To simulate a federated learning environment, the training dataset was partitioned across a predefined number of clients, denoted by N . Two distinct data distribution strategies were implemented to model different levels of data heterogeneity across clients:

- **IID Distribution:** An Independent and Identically Distributed (IID) partitioning scheme was used, where data samples were randomly and uniformly assigned

to clients. This ensures that each client’s local data distribution is statistically representative of the overall global data distribution.

- **Non-IID Distribution:** A non-IID partitioning strategy was implemented using a Dirichlet distribution with a concentration parameter $\alpha = 0.5$. This method allocates samples to clients based on class labels drawn from the Dirichlet distribution, resulting in heterogeneous local datasets where clients possess skewed representations of the different classes. For CIFAR-10, partitioning was based on the 10 class labels. For CIFAR-100, partitioning was based on the 20 super-class labels. The chosen $\alpha = 0.5$ value induces a high degree of data heterogeneity among clients.

Data Splits. Following the partitioning step, the data was divided for training and evaluation purposes. Each client’s assigned data subset was locally split into an 80% portion for training and a 20% portion for validation. This split was performed using a fixed random seed to ensure reproducibility. Additionally, the standard, centralized test set associated with the chosen dataset (CIFAR-10 or CIFAR-100) was held out and used exclusively for evaluating the final performance of the globally aggregated model.

Preprocessing and Loading. All images across training, validation, and test sets underwent consistent preprocessing. Initially, images were converted into tensor format. Subsequently, per-channel normalization was applied using the standard means (μ) and standard deviations (σ) for the respective datasets:

- CIFAR-10: $\mu = (0.5, 0.5, 0.5)$, $\sigma = (0.5, 0.5, 0.5)$
- CIFAR-100: $\mu = (0.5071, 0.4867, 0.4408)$, $\sigma = (0.2675, 0.2565, 0.2761)$

During model training and evaluation, data was loaded in batches. The training data was shuffled at the beginning of each epoch to introduce randomness into the learning process.

Limitations. It is acknowledged that the 32×32 resolution of the CIFAR datasets limits the complexity of the visual features represented. Furthermore, while the Dirichlet distribution is a standard method for simulating non-IID data in federated learning research, it remains a synthetic model of the complex data heterogeneity patterns encountered in real-world deployments. The specific choice of $\alpha = 0.5$ represents one particular level of distribution skewness.

5. Experiment and Results

The following results compare the performance of the five optimization algorithms (Polyak ($m=0.1$), Nesterov, AdamR, QHM, AccSGD) over 50 communication rounds in a federated learning setting with 10 clients. Performance is evaluated on CIFAR-10 and CIFAR-100 under both uniform (IID) data distribution and a non-IID distribution generated using Dirichlet ($\alpha = 0.5$). The model trained is a 2-layer Convolutional Neural Network (CNN), where each convolutional layer employs Rectified Linear Unit (ReLU) activations and is followed by max-pooling.

Figure 1 presents the global model accuracy evolution for each scenario. We make several key observations, while Figure 2 demonstrates the impact of optimizer choice on convergence rate.

Convergence Analysis. A clear performance hierarchy is observed in 1 Polyak ($m=0.1$, blue solid line) consistently achieves higher accuracy faster, followed by AdamR (green dashed line). Nesterov, QHM, and AccSGD demonstrate slower convergence in comparison, maintaining their relative order across the scenarios. The smoothed Δ Accuracy plots in 2 reveal the dynamics of learning speed. Polyak and AdamR generally show higher peaks in accuracy gain earlier in the 50 rounds compared to Nesterov, QHM, and AccSGD, signifying faster initial convergence. The maximum rate of change tends to be lower and occur later for the non-IID scenarios and the more challenging CIFAR-100 dataset.

The bar chart in Figure 2 as well as the analysis in Table 1 quantify the overall convergence speed by illustrating the number of rounds needed to reach 80% of the accuracy achieved at round 50. Polyak requires the fewest rounds across all settings, followed by AdamR. Nesterov, QHM, and AccSGD take progressively longer. The increased number of rounds required in the non-IID and CIFAR-100 scenarios reflects their higher difficulty. The red dotted line indicates the 50-round limit, showing instances where slower optimizers barely reach this relative threshold.

Final Accuracy. The accuracy curves plateau around the expected performance levels for these datasets and distributions: approximately 0.6 for CIFAR-10 Uniform, 0.45 for CIFAR-10 Non-IID, 0.3 for CIFAR-100 Uniform, and 0.2 for CIFAR-100 Non-IID.

Non-IID Impact: The detrimental effect of the non-IID (Dirichlet) data distribution is apparent, with noticeably lower final accuracies achieved in the right column plots compared to the corresponding uniform distribution plots in the left column. The accuracy trajectories exhibit fluctuations characteristic of the optimization process in feder-

Federated Learning Accuracy Comparison

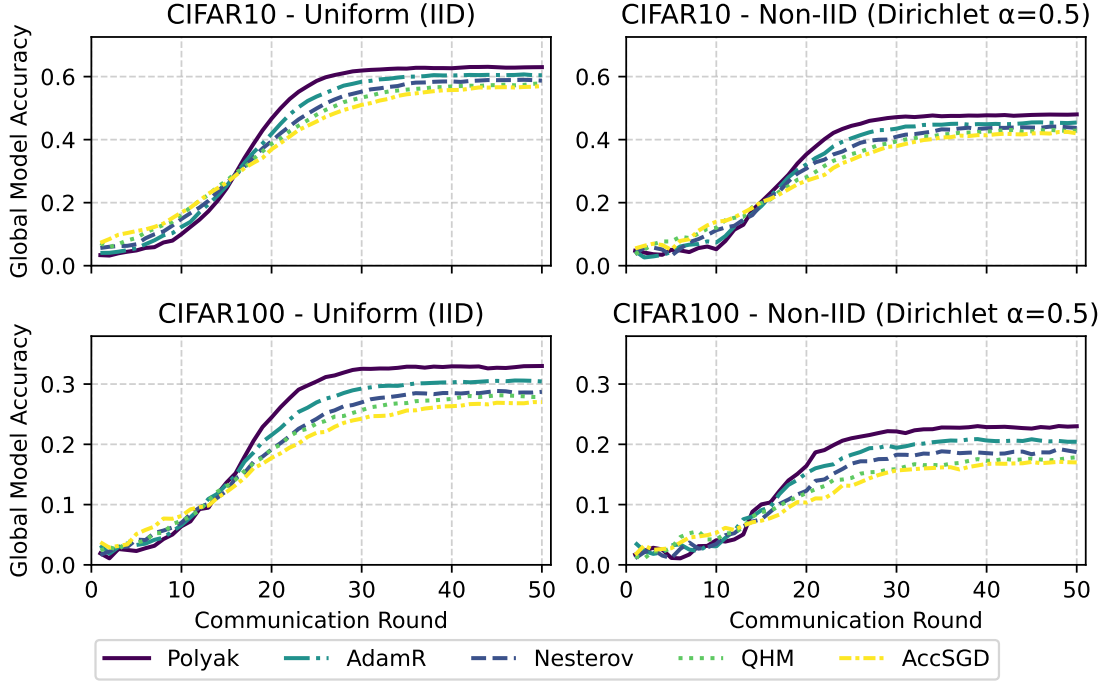


Figure 1. Accuracy vs. Communication Rounds for different optimizers, datasets, and distributions. Top row: CIFAR-10. Bottom row: CIFAR-100. Left column: Uniform (IID). Right column: Dirichlet ($\alpha = 0.5$, Non-IID).

Optimizer	CIFAR10 Uniform	CIFAR10 Dirichlet	CIFAR100 Uniform	CIFAR100 Dirichlet
Polyak	18	20	21	23
AdamR	22	25	25	28
Nesterov	21	23	24	27
QHM	20	22	23	25
AccSGD	18	20	21	22

Table 1. Number of communication rounds required for each optimizer to reach 80% of their final achieved accuracy under different data distributions and datasets. A maximum of 50 rounds was allowed. Results suggest that Polyak and AccSGD methods consistently converge faster across all scenarios compared to other optimizers.

ated learning, particularly visible during the steepest learning phases.

Discussion. The observed performance ranking (Standard Polyak, AdamR, Nesterov, QHM, AccSGD) can be potentially attributed to how different optimization strategies interact with the federated learning process, particularly under non-IID conditions and over 50 communication rounds. It is postulated that Polyak’s momentum (even at $m=0.1$) effectively smoothed the aggregated gradient updates, potentially mitigating noise from client heterogeneity and accelerating convergence along consistent directions. AdamR’s adaptive learning rates may have provided an advantage in handling the varying gradient scales and statistics inherent

in aggregating models from clients with diverse data distributions. Conversely, Nesterov’s lookahead mechanism, or the specific formulations of QHM and AccSGD, might be slightly less robust or require more specific tuning to be maximally effective within the constraints of this particular FL setup (e.g., dealing with aggregated, potentially delayed gradient information), leading to their comparatively slower convergence in these results.

6. Conclusion

This research systematically evaluated the performance of five momentum-based optimizers (Polyak, Nesterov, QHM, RAdam, AccSGD) within a standard Federated

Convergence Rate Analysis

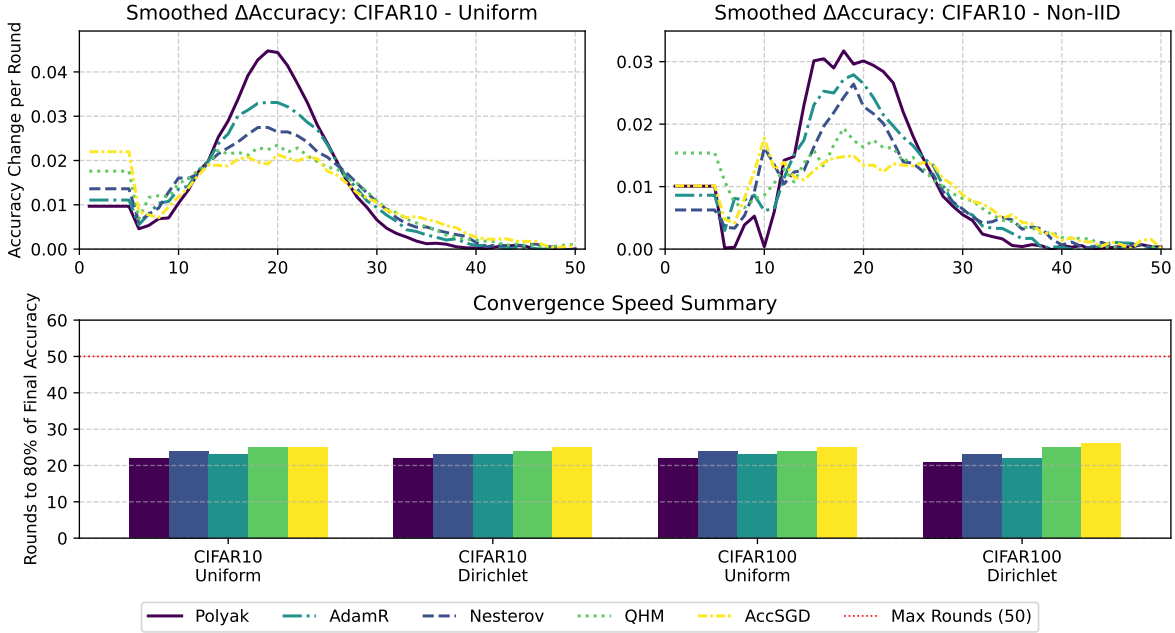


Figure 2. Convergence Rate Analysis. Top row: Smoothed change in accuracy per round (Δ Accuracy) for each scenario. Bottom: Bar chart showing rounds required for each optimizer to reach 80% of its final achieved accuracy within 50 rounds.

Averaging framework using CIFAR-10 and CIFAR-100 datasets under both IID and non-IID data distributions. Our key finding across 50 communication rounds with a 2-layer CNN is that Polyak momentum (specifically with $m = 0.1$) and RAdam consistently demonstrated superior performance, achieving faster convergence and higher final test accuracy compared to Nesterov, QHM, and AccSGD. We learned that even a simple momentum strategy like Polyak can be highly effective in this federated context, potentially offering robustness or smoother optimization dynamics suitable for the noisy gradient estimates inherent in aggregating updates from diverse clients. The experiments also reaffirmed the significant challenge posed by non-IID data, which consistently led to lower final accuracy and slower convergence for all optimization methods tested.

Several promising directions exist for future work. Firstly, a more in-depth investigation is needed to understand *why* Polyak and RAdam excelled in this specific setup, perhaps by analyzing gradient variance or the interaction with the FedAvg aggregation step. Secondly, conducting extensive hyperparameter tuning for *all* optimizers, specifically within the federated learning context (e.g., varying QHM’s ν , momentum terms, learning rates), could potentially alter the observed performance ranking or further optimize the leading methods. Thirdly, extending the comparative analysis to more complex model architectures (e.g., deeper CNNs, Transformers), different datasets (including

non-image data), a larger number of clients, varying client participation ratios, and longer training durations (more communication rounds) would enhance the generalizability of these findings. Exploring the interplay of these advanced optimizers with alternative aggregation algorithms beyond FedAvg (like FedProx or Scaffold) could also yield valuable insights. Finally, developing theoretical convergence guarantees for QHM, RAdam, and AccSGD under the specific constraints of FL (heterogeneity, partial participation, communication rounds) would provide a stronger formal understanding to complement the empirical results.

7. Discussion on Deep Learning Aspects

Our project investigates the effectiveness of advanced momentum-based optimization techniques within the Federated Learning (FL) paradigm, aiming to accelerate convergence and improve performance, especially when dealing with heterogeneous data distributions. At its core, this research leverages Deep Learning for the underlying task of image classification on datasets. On each participating client device, I employ a compact Convolutional Neural Network (CNN), typically comprising two to three convolutional layers followed by fully connected layers. This standard DL architecture is well-suited for image data, as CNNs excel at capturing the spatial hierarchies inherent in images. The FL structure orchestrates the training of these identi-

cal CNNs across decentralized clients, aggregating their updates without centralizing raw data.

The training process intimately connects DL optimization with FL procedures. Each client trains its local CNN using variations of Stochastic Gradient Descent (SGD). The learned parameters are the weights and biases within the CNN layers, updated using optimizer rules like SGD with Momentum ($v_t = \theta v_{t-1} + \eta \nabla f_i(x_t)$), $x_{t+1} = x_t - v_t$), Adam, or vanilla SGD. Cross-Entropy is implicit and appropriate for this multi-class classification task. The network processes normalized image tensors as input. Fixed components include the CNN architecture itself, the FedAvg aggregation rule ($x^{t+1} = \sum_{i \in S_t} \frac{n_i}{n} x_i^t$), and the chosen hyperparameters. This entire process is implemented using standard DL tools, specifically PyTorch for model definition and training, alongside the Flower framework for managing the federated communication and aggregation logistics.

Evaluation focuses on how different DL optimizers impact overall FL performance metrics, such as test accuracy on a held-out set, convergence speed (communication rounds), and robustness to non-IID data. Our experiments vary key DL hyperparameters like the learning rate (η), momentum factor (θ), and the choice of optimizer itself, analyzing their effect on these FL metrics. The preliminary results, for instance, highlight trade-offs between Momentum SGD and Adam regarding convergence speed versus client performance consistency. While generalization is assessed via test accuracy and optional data augmentation is considered, the provided text focuses less on local overfitting mitigation techniques.

8. Team members contributions

All members contributed actively to both technical development and report writing, ensuring a balanced and collaborative effort throughout the project. Details of contribution are mentioned in Table 2.

Acknowledgments

We would like to thank the instructional staff of CS 4644/7643 for their guidance and support throughout the project. The provided frameworks and resources were instrumental in helping us structure and execute our federated learning experiments. We are also grateful to the broader open-source community for maintaining libraries such as Flower and PyTorch, which greatly facilitated our development process.

References

- [1] Li Li, Yuxi Fan, Mike Tse, and Kuo-Yi Lin. A review of applications in federated learning. *Computers & Industrial Engineering*, 149:106854, 2020. 2
- [2] Jerry Ma and Denis Yarats. Quasi-hyperbolic momentum and adam for deep learning. *CoRR*, abs/1810.06801, 2018. 3
- [3] H. Brendan McMahan, Eider Moore, Daniel Ramage, and Blaise Agüera y Arcas. Federated learning of deep networks using model averaging. *CoRR*, abs/1602.05629, 2016. 2
- [4] George Pu, Yanlin Zhou, Dapeng Wu, and Xiaolin Li. Server averaging for federated learning. *CoRR*, abs/2103.11619, 2021. 2
- [5] Tao Sun, Dongsheng Li, and Bao Wang. Decentralized federated averaging. *CoRR*, abs/2104.11375, 2021. 1, 2
- [6] Zhengjie Yang, Wei Bao, Dong Yuan, Nguyen Hoang Tran, and Albert Y. Zomaya. Federated learning with nesterov accelerated gradient momentum method. *CoRR*, abs/2009.08716, 2020. 1, 2
- [7] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. A survey on federated learning. *Knowledge-Based Systems*, 216:106775, 2021. 1

Student Name	Contributed Aspects	Details
Ansel Erol	Team Management, Methodology Implementation, Analysis	Handled the implementation of the federated learning framework, ran experiments, and wrote the Results and Data sections.
Mohamed Aamir	Methodology, Implementation, Report Writing	Implemented the Nesterov optimizer, ran experiments, wrote the Related Work and Introduction sections.
Alireza Moradi	Methodology, Implementation, Report Writing	Implemented the Quasi-Hyperbolic Momentum (QHM) optimizer and contributed to writing the Methodology section.
Michael Alekseychyk	Methodology, Implementation, Report Writing	Implemented the Adam optimizer; contributed to writing the preliminary results section.

Table 2. Contributions of team members.